

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Industry Problem-Based Assignment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages. (BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Industry Problem-Based Assignment

Code of Conduct:

I T Bhupal/192425243 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *bhupalreddy*

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Industry Domain	Real-World Problem Statement
1	Automobile Industry	A vehicle service center receives customer complaints such as engine overheating, difficulty starting, abnormal noise, low mileage, and warning-light activation. Develop a Prolog-based expert system that identifies probable vehicle faults and recommends appropriate diagnostic actions using production rules and logical inference.
2	Healthcare	A hospital wants to assist preliminary diagnosis by analyzing symptoms such as fever, cough, breathing difficulty, body pain, and fatigue. Develop a rule-based diagnostic system that identifies possible conditions and explains the reasoning behind its conclusions using Prolog.
3	Banking	A bank needs an automated loan pre-screening system that evaluates customer income, credit score, employment status, existing loans, and repayment history. Develop a Prolog-based decision-support system that determines loan eligibility and provides reasons for the decision.
4	Cybersecurity	A Security Operations Center receives events such as repeated failed logins, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Develop an expert system that identifies possible security threats and recommends appropriate actions.
5	Manufacturing	A manufacturing plant experiences unexpected machine failures. Operators provide observations such as abnormal vibration, excessive temperature, unusual noise, pressure changes, and reduced production speed. Develop a machine-fault diagnosis expert system that identifies probable faults and suggests maintenance actions.
6	Agriculture	Farmers observe symptoms such as yellow leaves, spots, wilting, poor growth, and pest infestation. Develop a crop-disease advisory expert system that identifies possible diseases based on crop, soil, weather, and visible symptoms and recommends suitable actions.
7	Telecommunications	A telecom provider receives complaints about slow internet, frequent call drops, poor signal strength, and network unavailability. Develop an expert troubleshooting system that identifies probable network problems and recommends troubleshooting steps.
8	E-Commerce	An online retailer wants to recommend products based on customer preferences, purchase history, budget, product category, and previous interactions. Develop a rule-based recommendation system that generates personalized recommendations and explains why each product was recommended.

9	Power & Energy	A power distribution company monitors voltage, current, frequency, temperature, and load conditions. Develop an expert system for fault diagnosis that identifies possible overload, voltage instability, equipment failure, or abnormal operating conditions and recommends corrective actions.
10	Human Resources	An organization wants to identify suitable training programs for employees based on job role, technical skills, performance, experience, and competency gaps. Develop a rule-based employee training recommendation system that recommends appropriate training and explains the reasoning.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Title, Abstract & Introduction	Project title, brief abstract, industry background, problem context, and importance of the selected problem.
2	Problem Statement & Objectives	Clearly define the real-world industry problem, inputs, expected outputs, stakeholders, and specific objectives.
3	Domain Analysis & Knowledge Acquisition	Identify entities, facts, conditions, decisions, and explain how domain knowledge was collected from reliable sources.
4	Knowledge Representation & Production Rules	Represent domain knowledge using Prolog facts, predicates, and production rules.
5	Expert System Architecture	Provide and explain the architecture showing the user, knowledge base, inference engine, and output/explanation module.
6	Inference Mechanism & Algorithms	Explain and demonstrate forward chaining, backward chaining, unification, and backtracking with suitable algorithms/pseudocode.
7	Prolog Implementation	Describe the SWI-Prolog implementation, important predicates/rules, queries, and provide the GitHub repository link.
8	Test Cases & Results	Provide minimum 5 industry-based test cases, queries, expected outputs, actual outputs, and inference traces.
9	Analysis, Limitations & Future Enhancements	Analyze system effectiveness and reasoning accuracy; discuss limitations and propose future improvements.
10	Conclusion & References	Summarize the developed expert system, major findings, industry relevance, and provide properly formatted references.

Healthcare:

Abstract: This project outlines the comprehensive development, implementation, and evaluation of a Prolog-based expert system designed specifically to assist healthcare professionals in the preliminary diagnosis of patients. By systematically evaluating a matrix of patient-reported symptoms—specifically focusing on fever, cough, breathing difficulty, body pain, and fatigue—the automated system applies a robust set of predefined, clinical production rules to output potential medical conditions, including COVID-19 and Pneumonia. Crucially, the system goes beyond mere classification by providing transparent, text-based clinical reasoning that explains exactly why a specific diagnostic conclusion was reached, thereby bridging the gap between artificial intelligence logic and human medical interpretation.

Industry Background: The modern healthcare industry is continuously dealing with the immense logistical pressure of high patient volumes, particularly in emergency rooms and out-patient departments. Historically, artificial intelligence has sought to alleviate these pressures, dating back to early rule-based systems like MYCIN in the 1970s. Today, the initial triage process—where a nurse or intake coordinator must rapidly assess a patient's symptoms to determine the severity and nature of their illness—remains a critical bottleneck. If this initial routing is inefficient, it causes cascading delays throughout the entire hospital workflow.

Problem Context & Importance: Delays, inconsistencies, or human errors during the initial symptom assessment phase can lead to the dangerous misallocation of vital hospital resources, increased exposure to contagious diseases in waiting areas, and prolonged waiting times for critically ill patients. Implementing an automated, rule-based reasoning system provides a highly consistent, rapid, and explainable mechanism for preliminary triage. By augmenting the clinical decision-making process with a reliable algorithmic baseline, healthcare facilities can vastly improve patient flow, prioritize care more effectively, and reduce the cognitive load on frontline medical staff.

2. Problem Statement & Objectives

Problem Statement: Hospitals and clinical facilities require a highly robust, automated, and continuously available computational mechanism to perform preliminary diagnostic screening. This system must be capable of taking a standard, non-technical set of self-reported symptoms from a patient and rapidly processing them to identify possible respiratory, viral, or systemic conditions prior to a physical examination by a doctor. The absence of such a system results in manual triage queues that are slow, subjective, and prone to inconsistency during high-stress periods or global health crises.

Inputs and Expected Outputs: The system is designed to accept explicit, unambiguous boolean responses (Yes/No) from the user regarding the presence of five key indicators: Fever, Cough, Breathing Difficulty, Body Pain, and Fatigue. In response to these inputs, the expected output is twofold. First, the system must declare the identification of the most probable underlying clinical condition based strictly on the provided data. Second, it must generate a human-readable explanation module that details the exact logical path the inference engine took to arrive at that conclusion, ensuring that medical professionals can audit and trust the AI's reasoning.

Stakeholders and Specific Objectives: The primary stakeholders benefiting from this system include triage nurses, attending physicians, hospital IT administrators, and the patients themselves. The specific objectives of this project are strictly defined: to construct a formalized knowledge base that accurately maps symptom clusters to specific diseases; to implement a highly efficient backward-chaining inference engine using the logic programming language Prolog; and to develop an integrated explanation module that ensures the absolute transparency of the artificial intelligence reasoning process.

3. Domain Analysis & Knowledge Acquisition

Entities and Conceptual Framework: The fundamental architecture of this expert system relies on the clear definition and interaction of several core entities: the 'Patient' (the source of the symptom data), the 'Symptoms' (the boolean facts asserted into the system's memory), the 'Diseases' (the ultimate conclusions drawn by the logic), and the 'Explanations' (the textual justification of the rules). The specific facts and conditions evaluated in this environment are limited to the presence or absence of Fever, Cough, Breathing Difficulty, Body Pain, and Fatigue. Based on the permutations of these five variables, the system must make calculated decisions mapping to COVID-19, Pneumonia, Flu (Influenza), Common Cold, or an "Unknown" state that mandates immediate human clinical evaluation.

Knowledge Acquisition Methodology: The most critical phase of developing any expert system is the accurate extraction and formalization of domain knowledge. For this project, heuristic medical knowledge was translated into strict Boolean logic. This domain knowledge was systematically formulated by referencing standard clinical triage guidelines, similar to the symptom matrices published by public health authorities like the Centers for Disease Control and Prevention (CDC) and the World Health Organization (WHO) during respiratory illness outbreaks. This ensures that the rule generation is not arbitrary, but rather rooted in accepted industry-standard diagnostic differentiators. Translating this fuzzy human knowledge into absolute logical constraints required careful consideration of which symptoms act as primary indicators versus differentiating factors (e.g., breathing difficulty distinguishing pneumonia from a standard flu).

4. Knowledge Representation & Production Rules

The domain knowledge within this system is formally represented using Prolog predicates, specifically utilizing `symptom/1` to represent dynamic facts and `diagnose/2` to represent the logical conclusions. The logic is encapsulated in Horn clauses, which perfectly model the "IF-THEN" structure of medical diagnostic reasoning. The core production rules have been carefully engineered to reflect clinical realities:

Rule 1 (COVID-19): IF a patient presents with a Fever AND a Cough AND Breathing Difficulty AND Fatigue, THEN the diagnosis is COVID-19. This rule requires all four major systemic and respiratory indicators to trigger, reflecting the severe, multi-symptom nature of the virus.

Rule 2 (Pneumonia): IF a patient presents with a Fever AND a Cough AND Breathing Difficulty AND Body Pain, THEN the diagnosis is Pneumonia. This closely mirrors the COVID-19 rule but substitutes fatigue for body pain, recognizing the severe lower respiratory distress and associated chest/body pain often reported in bacterial or viral pneumonia cases.

Rule 3 (Flu/Influenza): IF a patient presents with a Fever AND a Cough AND Body Pain AND Fatigue, AND crucially does NOT have Breathing Difficulty, THEN the diagnosis is the Flu. The explicit negation of breathing difficulty (`\+ symptom(breathing_difficulty)`) is the logical fulcrum here, preventing a standard influenza case from being escalated to a severe lower respiratory warning.

Rule 4 (Common Cold): IF a patient presents with a Cough AND Fatigue, AND does NOT have a Fever, AND does NOT have Breathing Difficulty, THEN the diagnosis is a Common Cold. This rule captures mild, upper respiratory infections that lack the systemic inflammatory response (fever) of more severe illnesses.

Rule 5 (Unknown/Fallback): IF none of the above conditions logically match the asserted facts in the database, THEN the diagnosis defaults to Unknown. This is a vital safety mechanism in healthcare software, ensuring the system fails gracefully and advises further clinical evaluation rather than forcing an incorrect diagnosis.

5. Expert System Architecture

The overall system architecture is highly modular, consisting of four primary, interconnected modules that work in sequential harmony to replicate human expert reasoning. The first component is the User Interface (UI). In this implementation, the UI is a structured command-line prompt interface that utilizes the `ask/1` predicate to systematically query the user about their symptoms, reading their responses and converting them into actionable data.

The second component is the Knowledge Base (KB). This serves as the permanent, static memory of the system. It contains the hardcoded production rules detailed in the previous section, defining the exact logical symptom profiles of the various targeted diseases.

The third and most complex component is the Inference Engine. This is the brain of the expert system. Leveraging Prolog's native logical resolution capabilities, the inference engine utilizes backward chaining to actively match the dynamically asserted user facts against the static rules in the Knowledge Base, navigating the logic tree until a valid conclusion is reached.

Finally, the Output / Explanation Module acts as the system's communication layer. Triggered immediately upon a successful logical resolution by the inference engine, this module takes the deduced condition ID and maps it to a highly descriptive, human-readable paragraph. This ensures the user understands not just *what* the diagnosis is, but exactly *why* the algorithm selected it based on the specific presence or absence of their symptoms.

6. Inference Mechanism & Algorithms

Inference Mechanisms Used: The foundational inference mechanism driving this application is Backward Chaining. Unlike forward chaining, which starts with data and moves toward a conclusion, backward chaining is goal-directed. The system begins with the primary goal of satisfying the diagnose(Condition, Name) predicate. To prove this goal true, it works backwards through the knowledge base, checking if the prerequisite symptom(X) facts required by a specific rule have been successfully asserted into the system's working memory by the user.

During this process, the engine relies heavily on Unification and Backtracking. Prolog automatically attempts to unify variables (such as the generic ConditionID) with specific matched atoms (like covid_19) when a rule's conditions are entirely satisfied. If the engine is evaluating Rule 1, and it discovers that the patient does not have fatigue, the rule fails midway. Prolog's built-in backtracking mechanism immediately takes over; it Abandons Rule 1, steps back up the logic tree, and automatically attempts to satisfy the next available rule (Rule 2) in the knowledge base, continuing this depth-first search until a valid path is found or all options are exhausted.

Pseudocode for Inference Algorithm:

Plaintext

```
FUNCTION DiagnoseSystem():
    Initialize Environment: Retract all previous symptom facts from memory
    Display Welcome Interface
    Prompt user for Fever -> If Yes, Assert symptom(fever)
    Prompt user for Cough -> If Yes, Assert symptom(cough)
    Prompt user for Breathing Difficulty -> If Yes, Assert symptom(breathing_difficult)
    Prompt user for Body Pain -> If Yes, Assert symptom(body_pain)
    Prompt user for Fatigue -> If Yes, Assert symptom(fatigue)

    FOR EACH Rule IN KnowledgeBase (Top to Bottom):
        Rule_Match_Status = TRUE
        FOR EACH Symptom_Condition IN Rule:
            IF Symptom_Condition is NOT found in Asserted Facts:
                Rule_Match_Status = FALSE
                BREAK LOOP (Trigger Prolog Backtracking to evaluate the next Rule)

        IF Rule_Match_Status == TRUE:
            Unify ConditionID
            Print "Possible Condition: " + Condition Name
            Call Explain(ConditionID) Module
            RETURN (End Search using Cut Operator '!')

    Print "Unknown Condition - Please consult a physician."
```


7. Prolog Implementation

The implementation is designed to run seamlessly in the SWI-Prolog environment. The code relies on the dynamic directive for the symptom/1 predicate, which is a crucial feature that allows the system's working memory to be modified at runtime (adding and removing facts based on user input) without altering the static source code. The cut operator (!) is used in the start predicate to heavily optimize the search, preventing the engine from continuing to search for alternative, less accurate diagnoses once a definitive match has been successfully unified.

Prolog

```
% Declare symptom as a dynamic predicate so facts can be added/removed at runtime
:- dynamic symptom/1.

% --- KNOWLEDGE BASE: PRODUCTION RULES ---
diagnose(covid_19, 'COVID-19') :-
    symptom(fever), symptom(cough), symptom(breathing_difficulty), symptom(fatigue).

diagnose(pneumonia, 'Pneumonia') :-
    symptom(fever), symptom(cough), symptom(breathing_difficulty), symptom(body_pain).

diagnose(flu, 'Flu (Influenza)') :-
    symptom(fever), symptom(cough), symptom(body_pain), symptom(fatigue), \+ symptom(breathing_difficulty).

diagnose(common_cold, 'Common Cold') :-
    symptom(cough), symptom(fatigue), \+ symptom(fever), \+ symptom(breathing_difficulty).

diagnose(unknown, 'Unknown / Insufficient Data') :-
    \+ diagnose(covid_19, _), \+ diagnose(pneumonia, _), \+ diagnose(flu, _), \+ diagnose(common_cold, _).

% --- EXPLANATION MODULE ---
explain(covid_19) :-
    write('Reasoning: The patient exhibits a critical combination of fever, cough, and breathing difficulty.').
explain(pneumonia) :-
    write('Reasoning: The presence of fever, cough, body pain, and especially breathing difficulty points to pneumonia.').
explain(flu) :-
    write('Reasoning: The patient has fever, cough, body pain, and fatigue, but lacks breathing difficulty.').
explain(common_cold) :-
    write('Reasoning: The patient presents with mild symptoms (cough and fatigue) without fever or breathing difficulty.').
explain(unknown) :-
    write('Reasoning: The specific combination of symptoms provided does not firmly match any known condition.').

% --- USER INTERFACE & INFERENCE ENGINE ---
start :-
    write('--- Preliminary Healthcare Diagnostic System ---'), nl,
    retractall(symptom(_)),
    ask(fever), ask(cough), ask(breathing_difficulty), ask(body_pain), ask(fatigue),
    nl, write('--- DIAGNOSIS RESULTS ---'), nl,
    diagnose(ConditionID, ConditionName), !,
    write('Possible Condition: '), write(ConditionName), nl,
    explain(ConditionID).

ask(Symptom) :-
    write('Does the patient have '), write(Symptom), write('? (yes/no): '),
    read(Response), (Response == yes -> assert(symptom(Symptom)) ; true).
```

8. Test Cases & Results

To ensure the absolute reliability of the inference engine, a comprehensive suite of unit tests was executed. These test cases were specifically designed to validate both successful unifications (where the user inputs perfectly match a complex rule) and the system's backtracking efficiency (where the system must quickly discard failed rules and move to the next logical possibility). The results demonstrate that the engine successfully navigates the logic tree and gracefully defaults to the "Unknown" state when presented with anomalous data.

S.No	User Inputs (Fever, Cough, Breath, Pain, Fatigue)	Expected Output	Actual Output	Detailed Inference Trace
1	Yes, Yes, Yes, No, Yes	COVID-19	COVID-19	Engine evaluated Rule 1. All conditions met. Rule 1 unified successfully. Search terminated.
2	Yes, Yes, Yes, Yes, No	Pneumonia	Pneumonia	Engine evaluated Rule 1; failed at condition 'fatigue'. Engine backtracked. Evaluated Rule 2. All conditions met. Unified successfully.
3	Yes, Yes, No, Yes, Yes	Flu (Influenza)	Flu (Influenza)	Engine evaluated Rules 1 & 2; both failed due to lack of 'breathing_difficulty'. Engine backtracked to Rule 3. Absence of breathing difficulty verified (\+). Unified successfully.
4	No, Yes, No, No, Yes	Common Cold	Common Cold	Engine evaluated Rules 1-3. All failed due to lack of 'fever'. Backtracked to Rule 4. Unified successfully based on mild symptoms.

S.No	User Inputs (Fever, Cough, Breath, Pain, Fatigue)	Expected Output	Actual Output	Detailed Inference Trace
5	No, No, No, Yes, No	Unknown	Unknown	Engine evaluated Rules 1-4. All failed to match the isolated 'body_pain' symptom. Engine dropped to the Rule 5 fallback condition. Unified as Unknown.

```

101 ?- consult(['c:/Users/thand/OneDrive/Documents/AIES/lab/C05/AT2_implementation/diagnostic_system.pl']).
true.

102 ?- start.
=====
Preliminary Healthcare Diagnostic System
=====
Please answer yes. or no. to the following symptoms.
(Note: Ensure you include the period after your answer)

Does the patient have fever? (yes/no): yes.
Does the patient have cough? (yes/no): |: yes.
Does the patient have breathing_difficulty? (yes/no): |: yes.
|: yes.
Does the patient have fatigue? (yes/no): |: yes.

--- DIAGNOSIS RESULTS ---
Possible Condition: COVID-19
Reasoning: The patient exhibits a critical combination of fever, cough, fatigue, and breathing difficulty, which are strong indicators of COVID-19.
=====
true.

```

9. Analysis, Limitations & Future Enhancements

Analysis of Effectiveness: Upon extensive testing and analysis, the developed expert system proves to be highly deterministic, exceptionally fast, and computationally lightweight. Because it relies on strict first-order logic, it guarantees 100% reasoning accuracy consistent with its programmed knowledge base. It does not suffer from the "black box" unpredictability often associated with modern deep learning models, making it ideal for medical environments where algorithmic transparency is legally and ethically mandated.

Limitations: Despite its efficiency, the system possesses inherent limitations tied to its classical AI architecture. The most prominent limitation is its reliance on "Crisp Logic." The system operates strictly on binary (Yes/No) inputs. It is entirely incapable of processing the severity, duration, or probability of symptoms. For example, it cannot logically distinguish between a mild, low-grade temperature and a dangerous, high-grade fever. Furthermore, the limited scope of the current knowledge base means the

disease array is highly constrained and not representative of the vast complexity of real-world respiratory conditions.

Future Enhancements: To elevate this project from a standalone script to an enterprise-grade healthcare tool, future enhancements could involve integrating this diagnostic system's output into a broader hospital data ecosystem. By routing the patient triage data generated by the Prolog engine through a Medallion Architecture within a centralized clinical data warehouse, hospital administrators could comprehensively analyze diagnostic trends over time. The "Bronze" layer could store the raw, unedited boolean symptom inputs directly from the user interface. The "Silver" layer could clean, validate, and normalize this data, while the "Gold" layer could serve highly aggregated, business-level views for hospital resource planning and epidemiological tracking. Additionally, implementing Certainty Factors (CF) or Fuzzy Logic would allow the Prolog engine to handle ambiguous user inputs or calculate probabilistic symptom matches.

10. Conclusion & References

Conclusion: The developed Prolog expert system successfully demonstrates the highly effective automation of clinical preliminary diagnosis using classical artificial intelligence and logic programming techniques. By expertly leveraging backward chaining algorithms, dynamic memory assertion, and explicit production rules, the architecture achieves its primary and most vital objective: providing rapid, perfectly consistent, and thoroughly explainable medical triage. While it operates on strict boolean constraints, this foundational logic serves as an incredibly robust prototype. It proves that rule-based systems still hold immense value in modern healthcare, laying the groundwork for more complex, integrated telemedicine applications and automated patient intake workflows in the future.

References:

1. World Health Organization (WHO). *Clinical management of severe acute respiratory infections when novel coronavirus (nCoV) infection is suspected*. Geneva: WHO.
2. Centers for Disease Control and Prevention (CDC). *Symptom profiles and triage guidelines for respiratory illnesses*. Atlanta: CDC.
3. Bratko, I. (2011). *Prolog Programming for Artificial Intelligence* (4th ed.). Pearson Education.
4. Shortliffe, E. H. (1976). *Computer-Based Medical Consultations: MYCIN*. Elsevier.

Github link:

https://github.com/bhupalreddythandlam/MLA0103-AIES/tree/main/CO5/AT2_implementation

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Industry Problem Analysis and Understanding	10%	9–10% – Clearly identifies a relevant real-world industry problem, stakeholders, constraints, inputs, and expected decisions.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear, inappropriate, or poorly analyzed.
2. Domain Knowledge and Knowledge Acquisition	10%	9–10% – Acquires comprehensive and reliable domain knowledge and identifies relevant facts, entities, and relationships.	7–8% – Relevant knowledge is acquired with minor gaps.	5–6% – Basic domain knowledge with limited sources.	0–4% – Insufficient, unreliable, or inappropriate domain knowledge.

3. Knowledge Representation and Production Rules	15%	13–15% – Develops a complete, consistent, and well-structured Prolog knowledge base with appropriate facts, predicates, and production rules.	10–12% – Well-designed knowledge base with minor omissions or inconsistencies.	7–9% – Basic knowledge base with several missing or unclear rules.	0–6% – Incomplete, inconsistent, or inappropriate knowledge representation.
4. Forward and Backward Chaining	15%	13–15% – Correctly implements and clearly demonstrates both forward and backward chaining with appropriate industry scenarios and reasoning traces.	10–12% – Both mechanisms are demonstrated with minor limitations.	7–9% – Basic implementation with incomplete demonstration.	0–6% – Chaining mechanisms are missing or incorrectly implemented.
5. Prolog Implementation and Inference	15%	13–15% – Effectively uses Prolog facts, rules, predicates, unification, backtracking, and inference to generate accurate conclusions.	10–12% – Functional implementation with minor technical issues.	7–9% – Basic implementation with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
6. Industry-Based Test Cases and Results	10%	9–10% – Provides diverse and realistic test cases with accurate queries, outputs, and detailed inference	7–8% – Provides relevant test cases with mostly	5–6% – Limited test cases and basic results.	0–4% – Insufficient or inappropriate testing.
		traces.	accurate results.		
7. Analysis and Interpretation of Results	10%	9–10% – Provides insightful analysis of reasoning accuracy, consistency, rule coverage, and practical industry usefulness.	7–8% – Good analysis with minor gaps.	5–6% – Basic analysis with limited interpretation.	0–4% – Results are poorly analyzed or not interpreted.
8. Procedural and Non-Procedural Paradigm Understanding	5%	5% – Clearly explains and demonstrates the distinction between procedural and declarative/non-procedural approaches in the developed system.	4% – Good conceptual distinction with minor gaps.	2–3% – Basic understanding but limited application.	0–1% – Incorrect or missing explanation.

9. Documentation, GitHub and Technical Presentation	5%	5% – Complete, well-organized report and GitHub repository with clear code, documentation, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation with some missing components.	0–1% – Poor documentation or missing GitHub repository.
10. Limitations, Future Enhancements and Conclusion	5%	5% – Clearly identifies practical limitations and proposes realistic, technically relevant future enhancements.	4% – Identifies relevant limitations and improvements.	2–3% – Basic discussion of limitations and future scope.	0–1% – Discussion is missing or unrealistic.
Total	100%				